

Fundamentals of Web Applications

Backend Development: Data Persistence with SQLite

Lecture 23 – May 28, 2026

Michael Soprano – michael.soprano@uniud.it

University of Udine – Department of Mathematics,
Computer Science, and Physics (DMIF)

Outline

1. Persistent State and SQLite
2. Tables, Rules, and SQL
3. Express and the Storage Layer
4. Database-Backed Task API
5. Course Wrap-Up

State Beyond One Request

- A backend often manages **application state**
- Tasks, users, and messages should survive more than **one request**
- An in-memory **array** can store data only while Node.js is running
- Restarting the server resets data stored only in **memory**

```
const tasks = [  
  { id: 1, title: "Review backend slides", completed: false },  
  { id: 2, title: "Prepare REST exercise", completed: true }  
]
```

Persistence as a Storage Problem

- **Persistence** means that data remains available after the program stops
- Persistent data is stored outside the temporary memory of the **process**
- A backend usually stores long-lived state in a **database**
- Application code can read and change stored data across server **restarts**

Same API, Persistent Storage

- The public task API can keep the same **REST endpoints**
- The storage layer changes from an array to a **database**
- `GET /tasks` now reads rows from persistent storage
- `POST /tasks` now inserts a row that remains available after **restart**

Database Management Systems

- A **DBMS** stores and organizes structured data for applications
- It keeps data in a form that can be **queried** and **updated**
- It applies rules such as keys and constraints during data changes
- It separates application logic from low-level data **storage**

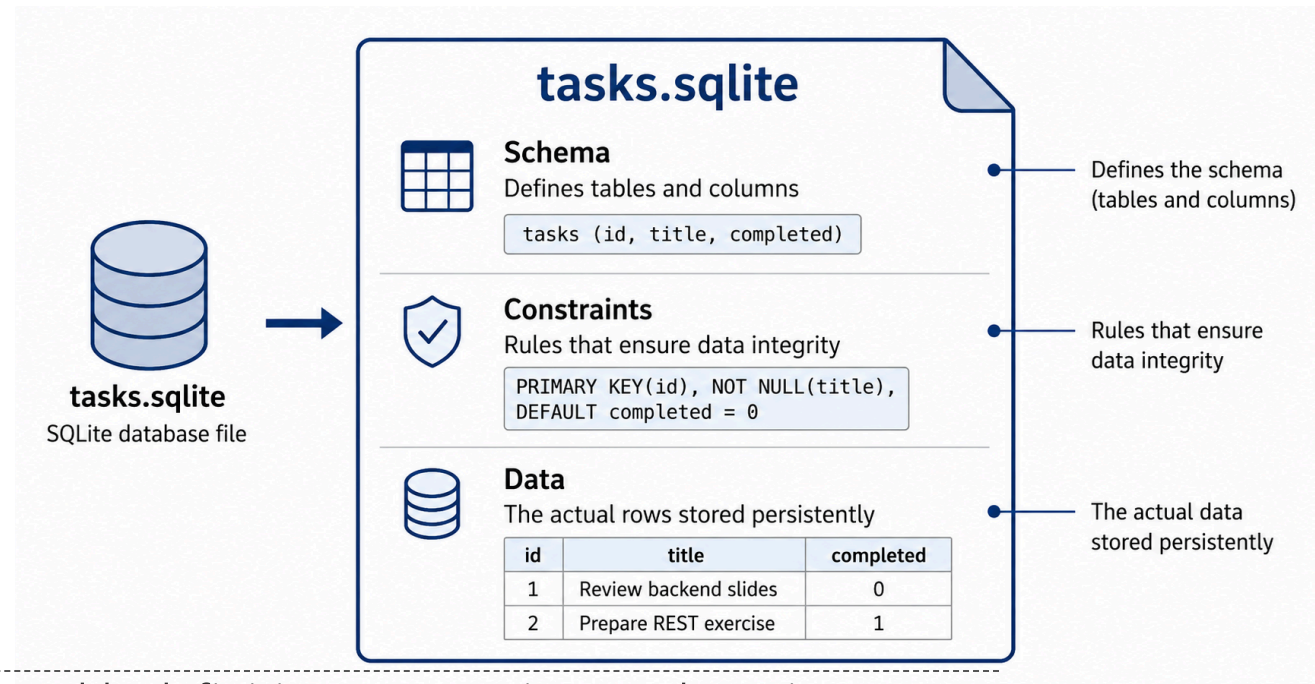
Embedded Database: SQLite

- **SQLite** is an embedded relational DBMS
- It does not require a dedicated database server to be running
- The application opens the database **file** directly
- This fits small applications, examples, prototypes, and local **development**



Database as a File

- In SQLite, a database is stored in a single **file**
- The file contains table structure, constraints, and data
- Copying the file copies the database **content**
- Opening the file gives the application access to the stored **state**



A SQLite database file stores table definitions, constraints, and persistent rows

SQLite for Small Backends

- SQLite keeps database setup **lightweight**
- The application reads and writes a local **database file**
- No separate database server or network configuration is required
- The database can be copied, replaced, or inspected during **development**

Additional Database Resources

- Here we focus on the database concepts needed for the **project**
- Additional slides are available in [Resources/](#)
- They cover relational databases in general and **SQLite** in more depth
- The material can be used as optional support while developing the project

Data Organized in Tables

- SQLite stores structured data in **tables**
- A table groups rows with the same **structure**
- Each row can represent one application **resource**
- Each column stores one property of that resource

Task Table

- The `tasks` table stores the persistent task **collection**
- Each row represents one task resource
- The `id` column identifies one specific **row**
- The `title` and `completed` columns store task **data**

Column	Role
<code>id</code>	Task identifier
<code>title</code>	Task text
<code>completed</code>	Completion state

Database Rules

- A database can enforce rules on stored **data**
- A **primary key** keeps each row identifiable
- Required values should not be missing
- Defaults can fill values that the server does not explicitly provide

```
id INTEGER PRIMARY KEY  
title TEXT NOT NULL  
completed INTEGER NOT NULL DEFAULT 0
```

SQL Operations

- A **query** is an instruction sent to the database
- SQL statements can read stored rows or request a change
- Backend code executes SQL inside the **storage layer**
- Each SQL operation corresponds to a kind of data access

SQL Operation	Main Effect
SELECT	Read stored rows
INSERT	Add new rows
UPDATE	Change existing rows
DELETE	Remove rows

SQL Statements for Tasks

- SQL statements express operations on stored **rows**
- The backend sends statements to SQLite through the storage **layer**
- Placeholders such as `?` receive values from JavaScript code
- `WHERE` selects the specific rows affected by the operation

Operation	SQL Statement
Read all	<code>SELECT id, title, completed FROM tasks</code>
Read one	<code>SELECT id, title, completed FROM tasks WHERE id = ?</code>
Create	<code>INSERT INTO tasks (title) VALUES (?)</code>
Update	<code>UPDATE tasks SET completed = ? WHERE id = ?</code>
Delete	<code>DELETE FROM tasks WHERE id = ?</code>

SQL Placeholders

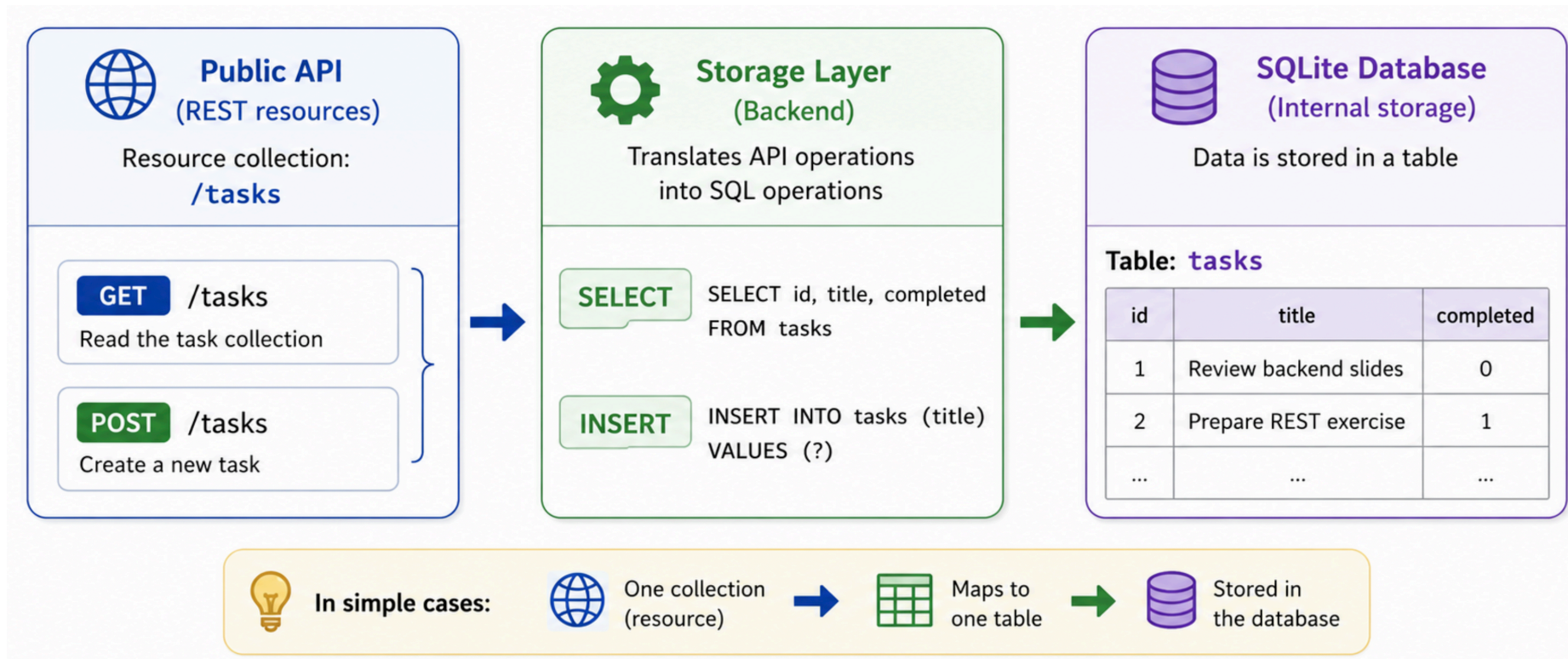
- SQL statements can contain **placeholders** for values decided at runtime
- The `?` placeholder marks a position where a JavaScript value will be inserted
- Values are passed in an **array**, separately from the SQL string
- The database driver matches placeholders and array values by **position**

Part	Role
SQL string	Contains the query and the <code>?</code> placeholder
<code>[taskId]</code>	Provides the value for the first <code>?</code>
<code>handleResult</code>	Handles the result when the operation finishes

```
database.get(  
  "SELECT id, title, completed FROM tasks WHERE id = ?",  
  [taskId],  
  handleResult  
)
```

API Resources and Tables

- REST resources belong to the public **API design**
- Database tables belong to the internal **storage design**
- In simple cases, one resource collection can map to one table
- The `tasks` resource can therefore be stored in a `tasks` table



Connecting Routes to Storage

- Express routes keep receiving **HTTP requests**
- The data source changes from memory to persistent **storage**
- The route should not contain every database detail
- A storage layer keeps HTTP logic and data access **separate**

Route Logic and Stored Data

- The route still interprets method, path, body, and **status codes**
- The handler no longer reads data from an in-memory **array**
- It calls a storage function to read or change persistent **data**
- The response still returns JSON to the frontend

Storage Layer

- The **storage layer** is the part of the backend that reads and writes data
- Moving data access into a separate module keeps routes easier to read
- `server.js` can focus on HTTP routes and **responses**
- `database.js` can focus on SQLite connections, table setup, and **queries**

SQLite Driver: `sqlite3`

- Node.js needs a package to communicate with SQLite
- The `sqlite3` package lets the backend open and query a SQLite database
- The dependency is installed with **npm**
- The package is recorded in `package.json`

```
npm install sqlite3
```

Database File Location

- SQLite stores the application database as a local **file**
- One database file can contain several **tables**
- Keeping the file in a `data` folder separates persistent data from source code
- The `data` folder must exist before SQLite can create the database file inside it

```
project-folder/  
├── data/  
│   └── app.sqlite  
├── public/  
├── database.js  
├── server.js  
└── package.json
```

Opening the Database File

- The database connection is created when the backend **starts**
- The connection points to the application SQLite **file**
- The same file can store multiple tables, such as `tasks`, `users`, or `messages`
- Route handlers should reuse the same connection through the storage **module**

```
const sqlite3 = require("sqlite3").verbose()

const database = new sqlite3.Database("data/app.sqlite")
```

Database File and Table Structure

- Opening a SQLite **file** gives access to the application database
- The file can contain several **tables**
- Each table defines rows for one kind of stored data
- The task API uses a `tasks` table inside the same database file

Creating the Task Table

- `CREATE TABLE` defines a new table inside the database
- The table name describes the stored **resource collection**
- Each column defines one stored property of the resource
- Constraints describe rules that SQLite should enforce

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  completed INTEGER NOT NULL DEFAULT 0  
)
```

Safe Schema Initialization

- Backend code can create the table when the application **starts**
- `IF NOT EXISTS` avoids failing when the table is already present
- Repeated server restarts keep the same table and stored **rows**
- Route code can then assume that the `tasks` table is available

```
database.run(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    completed INTEGER NOT NULL DEFAULT 0
  )
`)
```

Database Module Responsibilities

- A dedicated module opens the SQLite **database file**
- It initializes the required **table structure**
- It defines named data access **functions**
- It exports those functions to `server.js`
- Route handlers can use persistent data without knowing every SQLite detail

Reading Rows: `database.all()`

- `database.all()` executes a SQL query that can return several **rows**
- The first argument is the SQL **statement**
- The second argument is a function that runs when SQLite has finished
- That function receives an `error` or the returned **rows**

```
database.all(  
    "SELECT id, title, completed FROM tasks",  
    (error, rows) => {  
        if (error) {  
            return handleError(error)  
        }  
  
        handleError(null, rows)  
    }  
)
```

SQLite Result Arguments

- The SQLite result function follows an **error-first** pattern
- `error` contains information about a failed database operation
- `rows` contains the returned data when the query succeeds
- The data should be used only after checking that `error` is empty

Argument	Meaning
<code>error</code>	Database error, or <code>null</code> when the query succeeds
<code>rows</code>	Array of rows returned by the query

Result Handler in Storage Functions

- A storage function receives a result handler from the **route**
- The SQLite result is forwarded through that handler
- `handleResult(error)` reports that the database operation failed
- `handleResult(null, data)` reports that the operation succeeded

```
function getAllTasks(handleResult) {  
  database.all(  
    "SELECT id, title, completed FROM tasks",  
    (error, rows) => {  
      if (error) {  
        return handleResult(error)  
      }  
  
      handleResult(null, rows)  
    }  
  )  
}
```

Database Errors in Routes

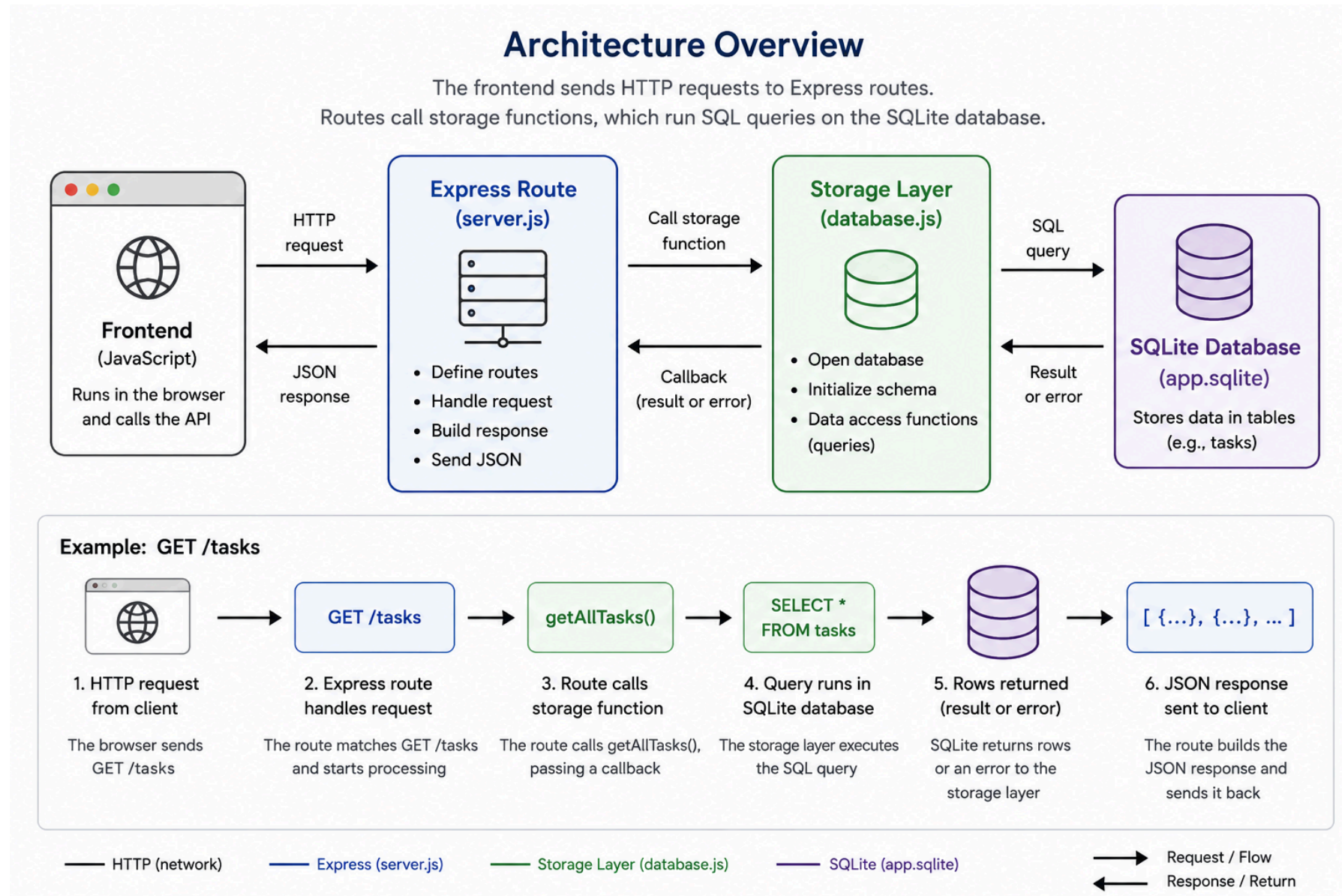
- The route checks the error received from the storage **function**
- Database errors are logged in the **terminal** during development
- The route sends a `500 Internal Server Error` response
- The request should never remain without a **response**

```
database.getAllTasks((error, tasks) => {  
  if (error) {  
    console.error(error)  
    return res.status(500).json({ error: "Database error" })  
  }  
  
  res.status(200).json(tasks)  
})
```

Storage Access Flow

- Express remains responsible for **HTTP**
- The storage layer manages database access through dedicated **functions**
- SQLite stores persistent data inside the application database **file**
- Results or errors move back through the same layers

Storage Access Architecture



Persistent Task API

- The task API keeps the same public **endpoints**
- Express serves the static frontend and handles API **routes**
- `database.js` manages SQLite access and storage functions
- `server.js` connects HTTP requests to persistent **data access**

Route and Storage Responsibilities

- The route handles HTTP methods, paths, bodies, and **status codes**
- The storage function executes SQL queries and returns **data**
- Database rows can be converted into API **representations**
- This boundary keeps HTTP logic separate from **storage logic**

Example: Source Code

- Sample **resource**: [AF_persistent-tasks/](#)
- The `data` folder is included in the project
- SQLite creates `app.sqlite` on the first server start
- `database.js` contains persistence logic
- `server.js` contains Express routes

```
AF_persistent-tasks/  
├── data/  
│   └── .gitkeep  
├── public/  
│   ├── index.html  
│   ├── style.css  
│   └── client.js  
├── database.js  
├── server.js  
└── package.json
```

Example: Database Module

- Sample **resource**: `AF_persistent-tasks/database.js`
- The module opens the application SQLite **database file**
- It creates the `tasks` table if needed
- The same file defines storage functions used by the **routes**

```
const sqlite3 = require("sqlite3").verbose()
const database = new sqlite3.Database("data/app.sqlite")

database.run(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    completed INTEGER NOT NULL DEFAULT 0
  )
`)
```

Example: Task Mapping

- Database rows are converted before leaving the **storage layer**
- SQLite stores `completed` as an integer
- The API exposes `completed` as a JavaScript **boolean**
- The frontend remains independent from SQLite storage details

```
function mapTask(row) {  
  return {  
    id: row.id,  
    title: row.title,  
    completed: Boolean(row.completed)  
  }  
}
```

Storage Function Shape

- A storage function wraps one database **operation**
- Input parameters describe the values needed by the SQL query
- The final parameter is a **result handler** supplied by the route
- The result handler receives either an error or the requested **data**

Part	Role
Input values	Values used by the SQL query
SQL operation	Work executed by SQLite
Result handler	Function called when the operation finishes

Task Storage Functions

- Each function gives the route a clear operation **name**
- The route passes only the values needed for that operation
- The storage layer hides SQL details from `server.js`
- Each function reports its result through `handleResult`

```
getAllTasks(handleResult)  
getTaskById(taskId, handleResult)  
createTask(title, handleResult)
```

Result Handler Pattern

- `handleResult(error)` reports a failed database operation
- `handleResult(null, data)` reports a successful operation
- The first argument is checked before using the returned data
- The route sends the HTTP response after the result handler runs

Result Handler Call	Meaning
<code>handleResult(error)</code>	The database operation failed
<code>handleResult(null, tasks)</code>	The operation returned a task array
<code>handleResult(null, task)</code>	The operation returned one task

Storage Function: `getAllTasks()`

- The function receives only the result **handler**
- `database.all()` reads all rows returned by the query
- Rows are mapped into task representations
- The result handler receives an array of **tasks**

```
function getAllTasks(handleResult) {  
  database.all(  
    "SELECT id, title, completed FROM tasks ORDER BY id",  
    (error, rows) => {  
      if (error) {  
        return handleResult(error)  
      }  
  
      handleResult(null, rows.map(mapTask))  
    }  
  )  
}
```

Storage Function: `getTaskById()`

- The function receives a task **identifier** and a result handler
- The identifier is passed to the SQL placeholder
- `database.get()` returns one row when a match exists
- The result handler receives one task, `null`, or an error

```
function getTaskById(taskId, handleResult) {  
  database.get(  
    "SELECT id, title, completed FROM tasks WHERE id = ?",  
    [taskId],  
    (error, row) => {  
      if (error) {  
        return handleResult(error)  
      }  
  
      handleResult(null, row ? mapTask(row) : null)  
    }  
  )  
}
```

Storage Function: `createTask()`

- The function receives an already validated **title** and a result handler
- The title is passed to the SQL placeholder
- `database.run()` executes the `INSERT` statement
- `this.lastID` provides the identifier assigned by SQLite

```
function createTask(title, handleResult) {  
  database.run(  
    "INSERT INTO tasks (title) VALUES (?)",  
    [title],  
    function(error) {  
      if (error) { return handleResult(error) }  
  
      handleResult(null, {  
        id: this.lastID,  
        title,  
        completed: false  
      })  
    }  
  )  
}
```

Exporting Storage Functions

- Sample **resource**: `AF_persistent-tasks/database.js`
- The module exports the functions needed by `server.js`
- Express routes do not need to know every SQLite detail
- The storage layer becomes a clear backend **boundary**

```
module.exports = {  
  getAllTasks,  
  getTaskById,  
  createTask  
}
```

Example: Server Setup

- Sample **resource**: `AF_persistent-tasks/server.js`
- Express serves the static frontend
- JSON request bodies are parsed with `express.json()`
- Storage functions are imported from `database.js`

```
const express = require("express")
const database = require("./database")

const app = express()
const port = 3000

app.use(express.static("public"))
app.use(express.json())
```

Route Uses Storage: Collection

- Sample **resource**: `AF_persistent-tasks/server.js`
- `GET /tasks` keeps the same collection **endpoint**
- The route calls `database.getAllTasks()`
- The result handler turns storage results into HTTP **responses**

```
app.get("/tasks", (req, res) => {  
  database.getAllTasks((error, tasks) => {  
    if (error) {  
      console.error(error)  
      return res.status(500).json({ error: "Database error" })  
    }  
  
    res.status(200).json(tasks)  
  })  
})
```

Route Uses Storage: Item

- Sample **resource**: `AF_persistent-tasks/server.js`
- `GET /tasks/:id` keeps the same item **endpoint**
- The route reads the `id` path parameter
- A missing task still becomes `404 Not Found`

```
app.get("/tasks/:id", (req, res) => {  
  const taskId = Number(req.params.id)  
  
  database.getTaskById(taskId, (error, task) => {  
    if (error) {  
      console.error(error)  
      return res.status(500).json({ error: "Database error" })  
    }  
  
    if (!task) {  
      return res.status(404).json({ error: "Task not found" })  
    }  
  
    res.status(200).json(task)  
  })  
})
```

Route Uses Storage: Creation

- Sample **resource**: `AF_persistent-tasks/server.js`
- The route validates the submitted **title**
- A valid request calls `database.createTask()`
- The created task is returned with `201 Created`

```
app.post("/tasks", (req, res) => {
  const submittedTitle = req.body.title
  if (typeof submittedTitle !== "string") { return res.status(400).json({ error: "Title is required" }) }

  const title = submittedTitle.trim()
  if (!title) {
    return res.status(400).json({ error: "Title is required" })
  }

  database.createTask(title, (error, task) => {
    if (error) {
      console.error(error)
      return res.status(500).json({ error: "Database error" })
    }
    res.status(201).json(task)
  })
})
```

Example: Static Document

- Sample **resource**: `AF_persistent-tasks/public/index.html`
- The form collects a new task **title**
- The list displays persistent tasks loaded from the API
- The status paragraph reports request errors

```
<main class="page">
  <h1>Persistent Tasks</h1>

  <form class="task-form">
    <label for="task-title">New task</label>
    <input id="task-title" name="title" required>
    <button type="submit">Add task</button>
  </form>

  <p class="status"></p>
  <ul class="task-list"></ul>
</main>
```

Example: Client Loading

- Sample **resource**: `AF_persistent-tasks/public/client.js`
- The frontend requests the stored task **collection**
- The JSON response is rendered as list items
- Helper functions and element selections are defined in the full **client script**

```
function loadTasks() {
  fetch("/tasks")
    .then(readJsonOrThrow)
    .then(showTasks)
    .catch(showError)
}

function showTasks(tasks) {
  taskList.replaceChildren()

  for (const task of tasks) {
    appendTask(task)
  }
}
```

Example: Client Creation

- Sample **resource**: `AF_persistent-tasks/public/client.js`
- The submit handler sends `POST /tasks`
- The request body contains the new task **title**
- The `Content-Type` header tells Express to parse the body as **JSON**

```
function createTask(title) {  
  return fetch("/tasks", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ title })  
  })  
}
```

Example: Client Submit Flow

- Sample **resource**: `AF_persistent-tasks/public/client.js`
- JavaScript intercepts the form **submission**
- The created task is rendered after the response arrives
- The input is cleared after a successful creation

```
function handleSubmit(event) {
  event.preventDefault()

  createTask(titleInput.value)
    .then(readJsonOrThrow)
    .then(task => {
      appendTask(task)
      titleInput.value = ""
    })
    .catch(showError)
}

taskForm.addEventListener("submit", handleSubmit)
loadTasks()
```

Example: Result

- The interface still behaves like the earlier task API
- New tasks are stored in the SQLite **database file**
- Restarting the server keeps previously created tasks available
- The frontend keeps using the same **JSON endpoints**

Persistent Tasks

Prepare final project

Add task

- Review backend slides
- Prepare REST exercise
- Prepare final project

Building Web Applications

- A web application connects **hypermedia documents**, interfaces, requests, and data
- HTML, CSS, JavaScript, HTTP, Express, and SQLite each solve a different part of the problem
- The main goal is understanding how the **pieces cooperate**
- The same principles remain visible in larger frameworks and production systems

Full-Stack Data Flow

- Browser JavaScript sends HTTP requests with **fetch**
- Express routes interpret methods, paths, headers, and bodies
- Storage functions read and write persistent **data**
- JSON responses return representations that the frontend can render
- Interface updates depend on the data returned by the backend

Frameworks Beyond the Course Stack

- Larger projects often use JavaScript frameworks to organize frontend **interfaces**
- Frameworks such as **Vue.js**, **React**, and **Svelte** structure components, state, and rendering
- Full-stack tools such as **Nuxt**, **Next.js**, and **SvelteKit** connect routing, rendering, and server-side code
- The same ideas remain underneath: requests, responses, state, data, and **interfaces**

Authentication and Sessions

- Many applications need to recognize which **user** is making a request
- Authentication checks user identity, often through a login **form**
- The backend verifies credentials and creates a session or token
- Later requests must carry some proof that the user is already **authenticated**

Cookies and User State

- A **cookie** is a small value stored by the browser for a specific site
- The browser can send that cookie automatically with later **requests**
- A session cookie can connect a request to server-side user **state**
- Cookies are central to many login systems, but they require careful security settings

Authorization and Protected Data

- Authentication says who the user **is**
- Authorization decides what the user is allowed to **access**
- Backend routes must check permissions before returning or changing protected data
- Frontend checks can improve the interface, but backend checks remain **necessary**

Deployment Concerns

- Deployment means running the application outside the local development **machine**
- The backend must run as a server process on a reachable **host**
- Configuration, environment variables, ports, domains, HTTPS, and logs become important
- Persistent data needs backups and a safe storage location

Production-Ready Applications

- Real applications need more than working routes and visible interfaces
- **Authentication, authorization**, validation, testing, logging, deployment, and backups become central
- Databases are chosen based on data model, scale, concurrency, and operational needs
- These concerns guide how a coursework prototype becomes a reliable **system**

Final Course Summary

- A full-stack application coordinates **frontend**, **backend**, and **storage**
- The frontend presents information and captures user interaction
- The backend applies application rules and exposes resource-oriented APIs
- Persistent storage keeps application state available across requests and restarts
- The final project combines these ideas into one working application

Bibliography

- Express. *Express Routing*. <https://expressjs.com/en/guide/routing.html>
- Express. *Express 5.x API Reference*. <https://expressjs.com/en/api.html>
- Node.js. *Node.js Documentation*. <https://nodejs.org/docs/latest/api/>
- npm. *npm Documentation*. <https://docs.npmjs.com/>
- MDN Web Docs. *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs. *HTTP response status codes*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>

Bibliography (cont.)

- Duckett, J. (2020). *PHP & MySQL: Server-side Web Development*. John Wiley & Sons
 - Chapters 4-6: Relational Databases, SQL Queries, Data Persistence
- SQLite. *SQLite Documentation*. <https://sqlite.org/docs.html>
- SQLite. *About SQLite*. <https://sqlite.org/about.html>
- SQLite. *SQLite In 5 Minutes Or Less*. <https://sqlite.org/quickstart.html>

Fundamentals of Web Applications

A.A. 2025/2026

Thank you for your attention

See you at the project discussion